Índice

- Objetivos de la sesión
- Conocimientos y materiales necesarios
- 1. *Pipeline* MIPS64
  - 1.1. Funcionamiento básico del *pipeline*
- 2. Riesgos de la segmentación
  - 2.1. Riesgos de datos
  - 2.2. Riesgos de control
  - 2.3. Riesgos estructurales
- Archivos de la práctica
- Ejercicios

Mejora del rendimiento con el *pipeline*

Área de Arquitectura y Tecnología de Computadores – Versión 2.1.12009, 25/09/2023

Objetivos de la sesión

En esta sesión se estudia el funcionamiento del *pipeline* en la microarquitectura segmentada en 5 etapas de los procesadores MIPS64. Los principales objetivos de esta sesión son:

- Consolidar los conceptos sobre el *pipeline* y los riesgos que producen roturas en el cauce de ejecución segmentado.
- Conocer algunas de las técnicas que permiten minimizar las roturas del cauce segmentado.

Conocimientos y materiales necesarios

Para poder realizar esta sesión, el alumno debe:

- Acudir al laboratorio con los apuntes de teoría correspondientes al tema dedicado a la CPU. Es importante además tener claras las ideas teóricas sobre el *pipeline*.
- Durante la sesión se plantearán una serie de preguntas que puedes responder en el correspondiente cuestionario en el campus virtual. Puedes abrir el cuestionario en otra pestaña del navegador pinchando en el enlace mientras mantienes pulsada la tecla `Ctrl`.

---

1. *Pipeline* MIPS64

- Descarga los ficheros necesarios para la práctica y descomprímelos en tu carpeta de trabajo.

Como sabes, la microarquitectura segmentada MIPS64 vista en clase organiza el cauce de ejecución en 5 etapas:

1. **IF**: búsqueda del código de instrucción e incremento del contador de programa.
2. **ID**: decodificación de la instrucción, lectura de registros y realización de saltos incondicionales.
3. **EX**: ejecución y cálculo de direcciones efectivas.
4. **MEM**: acceso a memoria y modificación del contador de programa en saltos condicionales.
5. **WB**: escritura diferida.

La ejecución segmentada de instrucciones aumenta la productividad de la CPU. En una situación ideal, el número de instrucciones por segundo que es capaz de ejecutar una CPU segmentada es mayor en un factor equivalente al número de etapas respecto a la versión secuencial (no segmentada) de la misma CPU. No obstante, existen riesgos en la segmentación que impiden que se alcance este máximo ideal. En las siguientes secciones veremos los distintos tipos de riesgos que pueden producirse en un cauce segmentado.

1.1. Funcionamiento básico del *pipeline*

Utilizaremos un pequeño programa para ilustrar el funcionamiento del cauce segmentado. Este programa, cuyo listado se muestra a continuación, realiza dos sumas de cuatro enteros almacenados en memoria. Al final, almacena ambos resultados en sendas posiciones de memoria.

```
    .data

var1: dword 14
var2: dword 24
var3: dword 5
var4: dword 4
res1: dword 0
res2: dword 0

    .code

main:
    ld    r10, var1(r0)  ; load var1 in r10
    ld    r20, var2(r0)  ; load var2 in r20
    ld    r11, var3(r0)  ;...
    ld    r21, var4(r0)
    nop                  ; nop = no operation
    dadd  r1, r20, r10   ; r1 <- r20 + r10
    dadd  r2, r21, r11
    nop
    sd    r1,res1(r0)    ; store r1 in res1
    sd    r2,res2(r0)
```

- Comprueba que el fichero `2-2ideal.asm` tiene el contenido arriba mostrado.
- Analiza el código fuente e intenta comprender su funcionamiento. La instrucción `ld r10, var1(r0)` carga en el registro `r10` el valor de `var1`. En este caso el registro `r0` se utiliza como registro base con valor cero. La instrucción `dadd` es la instrucción de suma para cantidades de 64 bits. La instrucción `sd r1, res1(r0)` almacena el contenido de `r1` en la variable `res1`. Observa que el orden del operando destino y fuente cambia en las operaciones de carga y almacenamiento.
- Suponiendo que existe una versión secuencial de la microarquitectura MIPS64, es decir, las 5 etapas en las que se divide el cauce de ejecución se ejecutan secuencialmente y no en paralelo, ¿cuántos ciclos serían necesarios para ejecutar el programa? Responde en el cuestionario: pregunta 1.
- Si esta CPU trabajase a una frecuencia de reloj de 400 MHz, ¿cuánto tiempo invertiría en ejecutar cada instrucción? Responde en el cuestionario: pregunta 2. ¿Cuál sería el tiempo necesario para completar el programa? Responde en el cuestionario: pregunta 3.
- ¿Cuál sería la productividad teórica medida en MIPS que puede alcanzar esta CPU? Responde en el cuestionario: pregunta 4.
- Considerando ahora el cauce segmentado donde las 5 etapas trabajan en paralelo, ¿cuál es la productividad máxima teórica de esta CPU en MIPS? Responde en el cuestionario: pregunta 5.
- Si se supone que ninguna de las instrucciones genera una detención, ¿cuál sería el número de ciclos necesarios para ejecutar el fragmento de código anterior? Responde en el cuestionario: pregunta 6.

Vamos a comprobar las respuestas utilizando el simulador. Comencemos primero con la simulación secuencial.

- Arranca el simulador.
- Carga en el simulador el fichero `2-2ideal.asm`.
- Haz clic en para establecer el tipo de microarquitectura a *Sequential* en la configuración de la microarquitectura.
- En este punto, el simulador está listo para iniciar el proceso de simulación secuencial. Mira la ventana principal y la ventana de `Source code` para observar la evolución de la simulación.
- Pulsa en ▶| para ejecutar la primera instrucción. Como la ejecución es secuencial, podrás ver que sólo hay una instrucción en el camino de datos en todo momento.
- Pulsa en ⏭ para simular todo el código fuente y comprobar tu respuesta anterior.

Hagamos ahora una simulación con la versión segmentada.

- Haz clic en para establecer el tipo de microarquitectura a *Pipelined* en la configuración de la microarquitectura.

  > ℹ La simulación se reinicia cada vez que cambian los ajustes de la microarquitectura.

- Utiliza el botón ▶ para ejecutar el primer ciclo de la primera instrucción. Observa cómo la ventana de `Source code` y la ventana principal muestran la primera instrucción entrando en el pipeline en el primer ciclo para ejecutar la etapa `IF`. Además, observa cómo la ventana *Pipeline* muestra la instrucción en la etapa `IF` y cómo el registro `IF/ID` contiene los datos de esta instrucción. El número representado en ambos componentes es el número de instrucción.
- Pulsa de nuevo el botón ▶ para simular un nuevo ciclo de reloj. La instrucción `ld r10, var1(r0)` pasa a la etapa `ID`, mientras que una nueva instrucción pasa a la primera etapa del pipeline. Esto también se puede ver claramente en la ventana *Pipeline*.
- Pulsa el botón ▶ otras tres veces. Deberías haber simulado el quinto ciclo. Durante el quinto ciclo todas las etapas del cauce segmentado están trabajando. Se ha superado un transitorio inicial desde que el cauce de ejecución estaba completamente vacío hasta que está completamente lleno. Este transitorio se produce cuando se reinicia la CPU. Al final del ciclo 5 terminará de ejecutarse la primera instrucción.
- Pulsa una vez más el botón ▶. Se ejecuta en este ciclo la última etapa de la segunda instrucción.
- Continúa la ejecución hasta el final pulsando el botón ⏭. Tras la ejecución de la última instrucción el simulador detendrá la ejecución. En ese momento el programa habrá terminado. ¿Cuántos ciclos se han consumido en la ejecución del programa? ¿Coincide con tu respuesta anterior?

Como has podido comprobar, el *pipeline* mejora significativamente el rendimiento. No obstante, hay situaciones en las que esa mejora ideal de rendimiento no se alcanza. En lo que resta de práctica se estudiarán los riesgos de la segmentación que impiden llegar a ese rendimiento máximo ideal.

---

2. Riesgos de la segmentación

Se conocen como riesgos de la segmentación aquellas situaciones que pueden impedir el funcionamiento en paralelo de las diferentes etapas en las que se divide un cauce de ejecución segmentado. En esos casos, la ejecución en paralelo se interrumpe momentáneamente y se dice que se produce una detención, teniendo que esperar unas etapas por otras, lo que ocasiona que la productividad final de un cauce segmentado se reduzca y sea inferior a la teórica.

En las clases de teoría se han visto algunas situaciones que producen detenciones de la segmentación. Se han ejemplificado sobre un diseño muy simple del cauce segmentado MIPS64 dividido en 5 etapas. Bajo estas condiciones, las detenciones en el cauce segmentado se producían por dos motivos: una dependencia de datos entre una o varias instrucciones dentro del cauce o un cambio en el flujo de ejecución del programa. A continuación se describen los dos escenarios anteriores junto con los riesgos estructurales, que no aparecen en el cauce segmentado simplificado (que solo soporta operaciones con enteros y un subconjunto de instrucciones MIPS64).

2.1. Riesgos de datos

Los riesgos de datos son aquellos que se producen porque una instrucción depende del resultado de otra instrucción que todavía sigue en el cauce. Vamos a verlo con un ejemplo.

```
    .data

var: dword 25
res: dword 0

    .code

main:
    daddi r10, r0, 5   ; r10 <- 5
    nop
    ld    r20, var(r0) ; load var in r20
    dadd  r1, r20, r10 ; r1 <- r20 + r10
    nop
    nop
    sd    r1, res(r0)  ; store r1 in res
```

- Comprueba que el fichero `2-2data.asm` tiene el contenido arriba mostrado. Analiza brevemente el código e intenta comprender su funcionamiento. Fíjate que es un programa muy simple, que realiza la suma de un entero con una constante almacenando el resultado en memoria.
- Si no existiesen detenciones en la segmentación, ¿cuántos ciclos de reloj invertiría la CPU en ejecutar el programa? Responde en el cuestionario: pregunta 7.
- Carga el programa en el simulador.
- Asegúrese de que se encuentra seleccionada la microarquitectura segmentada y de que no esté activada la opción de menú **Microarchitecture › Forwarding**.
- Pulsa el botón ▶ cinco veces, hasta que se muestre la quinta etapa de la primera instrucción.
- Fíjate que en este ciclo (etapa `ID` de la instrucción `dadd r1, r20, r10`) se leen los enteros almacenados en los registros `r20` y `r10`. Sin embargo, si bien el registro `r10` ya está cargado con el valor inmediato al final del ciclo 5 (el valor se escribe realmente en la primera mitad del ciclo en la etapa `WB` y se lee en la segunda mitad en la etapa `ID`), debe esperarse a que el registro `r20` tenga el valor que le corresponde, que ocurrirá en la etapa `WB` de la instrucción `ld r20, var(r0)`. Por este motivo, se produce una detención en la segmentación por dependencia de datos, y se refleja en el simulador con un borde negro en la etapa detenida. Pulsa el botón ▶ y compruébalo.
- En la ventana *Pipeline* también se puede ver que la instrucción está todavía en la etapa `ID` y que la `EX` está inactiva. Como comprobarás, no hay ninguna instrucción en la etapa `EX`.
- El valor que contendrá el registro `r20` tras el acceso a la memoria se escribe en la etapa `WB`, por lo que la instrucción de suma debe esperar a que la instrucción de carga llegue a esa etapa. Pulsa el botón ▶ dos veces y compruébalo.
- Continúa la ejecución hasta el final pulsando el botón ⏭. Comprueba que comprendes lo que sucede en el *pipeline* en cada ciclo. ¿Cuántos ciclos ha invertido la CPU realmente en ejecutar el programa? Responde en el cuestionario: pregunta 8. ¿Cuál es la aceleración obtenida en la ejecución del código anterior respecto a una ejecución ideal sin detenciones en el cauce segmentado? Responde en el cuestionario: pregunta 9. ¿Y respecto a una hipotética CPU MIPS64 secuencial? Responde en el cuestionario: pregunta 10.

El valor que contendrá el registro `r20` tras el acceso a la memoria se conoce tras la finalización de la etapa `MEM`, pues el valor leído de la memoria ya está disponible. Solo tras la finalización de la etapa de escritura (`WB`) este valor estará realmente almacenado en el registro destino. Sin embargo, la microarquitectura MIPS64 incorpora una funcionalidad que permite utilizar este valor como operando de una instrucción a pesar de que no esté todavía guardado en el registro destino, con lo que se evita otro ciclo de detención en el cauce. A esta técnica se la conoce como adelantamiento o *forwarding* y se estudiará en la siguiente práctica. Por el momento, supondremos que no se utilizan este tipo de técnicas.

2.2. Riesgos de control

Los riesgos de control se producen en situaciones de cambios en el flujo de ejecución tales como llamadas a función, saltos o excepciones. Bajo estas condiciones, se interrumpe el flujo de ejecución secuencial, lo que impide a la CPU ejecutar las etapas iniciales de las siguientes instrucciones hasta que se resuelva la dirección de destino. Vamos a ilustrarlo con un ejemplo.

```
.data

.code
main:
    xor   r10, r10, r10
    daddi r20, r0, 12
    daddi r21, r0, 3
    daddi r22, r0, 2
    j     dest          ; jump (unconditional)
    nop
    xor   r20, r20, r20
dest:
    dsub  r20, r20, r21
    dadd  r22, r22, r21
```

- Comprueba que el fichero `2-2contr-j.asm` tiene el contenido arriba mostrado y analiza su código. Se trata de un programa muy simple que contiene una instrucción de salto incondicional, `j dest`, que rompe el flujo de ejecución secuencial y avanza el contador de programa hasta la instrucción `dsub`.
- Si no existiesen detenciones en la segmentación, ¿cuántos ciclos de reloj invertiría la CPU en ejecutar el programa? Responde en el cuestionario: pregunta 11.
- Vamos a verificarlo con el simulador. Carga el programa `2-2contr-j.asm`.
- Observa la ventana de código. La columna de la izquierda es el desplazamiento desde el comienzo de la sección de código a partir de la que se encuentra cada instrucción. Recuerda que cada instrucción ocupa en memoria 4 posiciones.
- Pulsa el botón ▶ de forma repetida hasta que la instrucción `j dest` entre en la primera etapa (`IF`), es decir, hasta el ciclo 5. Al final de este ciclo se acaba de ejecutar la primera etapa de la instrucción, que consiste en la búsqueda en memoria del código de la instrucción.
- Fíjate en la vista del diagrama de ciclos de reloj y consulta el código fuente del programa. Pulsa de nuevo el botón ▶, verás que comienza a ejecutarse la etapa `ID` de la instrucción `j dest`, mientras que la instrucción `nop` pasa a la etapa `IF`. Esto es así porque el *pipeline* asume una ejecución secuencial de instrucciones. Sólo al final de esta etapa (ciclo 6) es cuando la CPU decodifica la instrucción `j dest` y detecta un salto en el flujo de ejecución.

  > ℹ Recuerda que los saltos incondicionales se toman en la etapa `ID`.

- Observa en la ventana *Pipeline* cómo el registro `IF/ID` del pipeline no está escrito. Esto significa que la etapa `IF` ha sido descartada.
- Pulsa de nuevo el botón ▶. La CPU ha identificado la instrucción de salto y ha modificado el contador de programa (el salto se realiza siempre). Así, se ejecuta la primera etapa de la instrucción destino del salto, es decir, `dsub r20, r20, r21`.
- Continúa la ejecución hasta el final pulsando el botón ⏭. Comprueba que comprendes lo que sucede en el *pipeline* en cada ciclo. ¿Cuántos ciclos ha invertido la CPU realmente en ejecutar el programa? Responde en el cuestionario: pregunta 12. ¿Cuál es la aceleración obtenida en la ejecución del código anterior respecto a una ejecución ideal sin detenciones en el cauce segmentado? Responde en el cuestionario: pregunta 13. ¿Y respecto a una hipotética CPU MIPS64 secuencial? Responde en el cuestionario: pregunta 14.

Vamos a considerar ahora el caso de los saltos condicionales. El siguiente programa muestra un ejemplo de bucle controlado por un salto condicional.

```
    .code

main:
    daddi r8, r0, 2     ; 2 iterations
    xor   r10, r10, r10
    nop
loop:
    daddi r8, r8, -1
    daddi r21, r0, 3
    daddi r22, r0, 2
    bnez  r8, loop      ; branch if r8 != 0
    xor   r20, r20, r20
```

- Comprueba que el fichero `2-2contr-bnez.asm` tiene el contenido arriba mostrado y analiza su código. Se trata de un programa que incluye un bucle de tipo *do-while* que itera dos veces. El bucle está controlado por la instrucción, `bnez r8, loop`, que realiza un salto hacia atrás en el caso de que el registro `r8` sea distinto de cero.
- Si no existiesen detenciones en la segmentación, ¿cuántos ciclos de reloj invertiría la CPU en ejecutar el programa? Responde en el cuestionario: pregunta 15.
- Vamos a verificarlo con el simulador. Carga el programa `2-2contr-bnez.asm`.
- Pulsa el botón ▶ de forma repetida hasta que la instrucción `bnez r8, loop` entre en la primera etapa (`IF`), es decir, hasta el ciclo 7. Al final de este ciclo se acaba de ejecutar la primera etapa de la instrucción, que consiste en la búsqueda en memoria del código de la instrucción.
- Fíjate en la vista del diagrama de ciclos de reloj y consulta el código fuente del programa. Pulsa de nuevo el botón ▶, verás que comienza a ejecutarse la etapa `ID` de la instrucción `bnez r8, loop`, mientras que la instrucción `xor` pasa a la etapa `IF`. En este momento, todas las instrucciones que vienen después del salto condicional están paradas como se puede ver en la ventana *Pipeline*.
- Pulsa de nuevo el botón ▶ tres veces. Como el valor de `r8` es distinto de cero, el salto se toma. Esto se calcula en la etapa `EX` y se efectúa en la etapa `MEM` al utilizar evaluación normal de saltos. Verás que el efecto es mayor que con el salto incodicional.

Vamos a ejecutar ahora el programa anterior utilizando la técnica de evaluación agresiva de saltos.

- Habilita la opción de evaluación agresiva de saltos en el menú **Microarchitecture › Early branch evaluation**
- Vuelve a ejecutar la simulación hasta que la instruccion `bnez r8, loop` entre en la primera etapa (`IF`), es decir, hasta el ciclo 7.
- Pulsa de nuevo el botón ▶, verás que comienza a ejecutarse la etapa `ID` de la instrucción `bnez r8, loop`, mientras que la instrucción `xor` pasa a la etapa `IF`.
- Pulsa de nuevo el botón ▶. Al igual que en la ejecución anterior, el salto se toma, pero en este caso la decisión se toma en la etapa `ID` al utilizar evaluación agresiva de saltos. Verás que el efecto, ahora, es el mismo que con el salto incondicional. De esta forma se evita la penalización que supone tomar la decisión de saltar en etapas tardías del *pipeline*, ya que no es necesario descartar un elevado número de instrucciones.
- Pulsa repetidamente el botón ▶ de nuevo hasta que la instrucción de salto condicional vuelva a la etapa `ID`. En este caso, el salto no se toma, pues `r8` vale cero.
- Pulsa dos veces más el botón ▶. Como el simulador no implementa, en este momento, ningún un predictor saltos, el *pipeline* se detiene un ciclo en la etapa `IF` de la instrucción `xor` para después continuar ejecutándola.
- Continúa la ejecución hasta el final pulsando el botón ⏭. Comprueba que comprendes lo que sucede en el *pipeline* en cada ciclo. ¿Cuántos ciclos ha invertido la CPU realmente en ejecutar el programa? Responde en el cuestionario: pregunta 16. ¿Cuál es la aceleración obtenida en la ejecución del código anterior respecto a una ejecución ideal sin detenciones en el cauce segmentado? Responde en el cuestionario: pregunta 17. ¿Y respecto a una hipotética CPU MIPS64 secuencial? Responde en el cuestionario: pregunta 18.

2.3. Riesgos estructurales

Los riesgos estructurales son aquellos que se producen porque varias de las instrucciones del cauce intentan utilizar un recurso que no está replicado. La implementación del cauce de ejecución MIPS64 simplificada impide que se produzcan detenciones por riesgos estructurales, ya que cada etapa utiliza recursos independientes del resto.

Esto no es así cuando se utilizan unidades funcionales que requieren varios ciclos. Un ejemplo es la ejecución de operaciones aritméticas sobre números en punto flotante sobre la unidad de punto flotante. El simulador nos permite especificar cuántos ciclos necesita la CPU para realizar una operación de punto flotante. En la práctica, definir un número de ciclos mayor que uno para las operaciones de punto flotante equivale a que cada operación de punto flotante requiere de varias etapas, pues habitualmente las unidades de punto flotante también se segmentan.

Se ilustrarán los riesgos estructurales de la segmentación utilizando un programa de ejemplo que trabaja con números en punto flotante, ya que en este caso sí se pueden producir detenciones por este tipo de riesgos.

```
.data
    var1:   double 3.5
    var2:   double 1.7
    var3:   double 2.8
    var4:   double 0.4
    res1:   double 0
    res2:   double 0

.code
main:
    l.d   f10, var1(r0)
    l.d   f20, var2(r0)
    l.d   f11, var3(r0)
    l.d   f21, var4(r0)
    nop
    add.d f0, f20, f10  ; floating-point addition f0 <- f20 + f10
    add.d f1, f21, f11
    nop
    nop
    s.d   f0, res1(r0)  ; store f0 in res1
    s.d   f1, res2(r0)
```

- Confirma que el fichero `2-2struc.asm` tiene el contenido arriba mostrado. Analiza brevemente el código e intenta comprender su funcionamiento.
- Si no existiesen detenciones en la segmentación, y suponiendo que la unidad de suma de números en punto flotante necesita el mismo tiempo para completarse que el resto de etapas, ¿cuántos ciclos de reloj invertiría la CPU en ejecutar el programa? Responde en el cuestionario: pregunta 19.
- Carga el programa en el simulador.
- Configura la microarquitectura activando la opción `out-of-order retirement`. De esta forma se permite la terminación de instrucciones fuera de orden.
- Configura la latencia de la unidad de suma de punto flotante (el número de etapas en la suma de punto flotante). Para ello, incluye en el menú **Microarchitecture › Non-pipelined multicycle ins** la instrucción `add.d`. Indica que la unidad de punto flotante tarda dos ciclos (`EXB` cycles con valor 2).
- Presta atención a la ventana *Pipeline*. Ahora, tenemos dos unidades funcionales en la etapa `EX`: la etapa `EX` regular usando la ALU, que requiere 1 ciclo de reloj, y la unidad de punto flotante, que requiere 2 ciclos de reloj.
- Activa la opción `Show instruction timeline details` del simulador para que muestre información adicional en el cronograma.

  > ℹ Como puedes ver, el simulador permite utilizar unidades de ejecución multiciclo segmentadas y no segmentadas para las diferentes instrucciones.

- Avanza la ejecución hasta que se muestre la etapa `ID` de la instrucción `add.d f0, f20, f10` en la ventana principal (ciclo #7). En este momento la instrucción se ha traído desde la memoria y se ha decodificado, por lo que se va a proceder a realizar la operación de suma utilizando la unidad de suma de punto flotante. Puedes ver en la ventana *Pipeline* que la instrucción se envía a la unidad de ejecución de punto flotante. Por eso los datos de la instrucción se copian en el registro del pipeline `ID/EXB` en lugar del registro `ID/EXA`.
- Pulsa el botón ▶ (ciclo #8). Comprobarás que comienza la ejecución del primero de los dos ciclos de reloj que necesita la unidad de suma de punto flotante. Al mismo tiempo, la siguiente instrucción de coma flotante se decodifica en la etapa `ID`, pero esta etapa no avanza al estar su registro de segmentación de salida, `ID-EXB`, ocupado con la instrucción anterior de suma (el valor del registro no puede cambiar hasta que concluya la suma). Por lo tanto, la segunda instrucción de suma sufre una detención estructural en `ID`.
- Pulsa el botón ▶ (ciclo #9). Verás que termina la etapa de ejecución de la primera instrucción de suma y por tanto la segunda instrucción de suma puede avanzar a través de la etapa `ID`.
- Pulsa el botón ▶ (ciclo #10). La segunda instrucción de suma entra en la primera etapa `EXB` y a la vez se decodifica en la etapa `ID` la instrucción `nop` y se envía a la unidad de ejecución `EXA`, por lo que sus datos se copian en el registro `ID/EXA`.
- Pulsa el botón ▶ (ciclo #11). En este ciclo tanto la instrucción `nop` como la segunda instrucción de suma intentan pasar a la etapa `MEM` pero como esta etapa no está replicada, la instrucción `nop` debe esperar en la etapa `EXA` produciéndose otra detención en la segmentación por un riesgo estructural.
- Continúa la ejecución hasta el final pulsando el botón ⏭. Comprueba que comprendes lo que sucede en el *pipeline* en cada ciclo. ¿Cuántos ciclos ha invertido la CPU realmente en ejecutar el programa? Responde en el cuestionario: pregunta 20. ¿Cuál es la aceleración obtenida en la ejecución del código anterior respecto a una ejecución ideal sin detenciones en el cauce segmentado? Responde en el cuestionario: pregunta 21. ¿Y respecto a una hipotética CPU MIPS64 secuencial? Responde en el cuestionario: pregunta 22.

Archivos de la práctica

En tu carpeta de trabajo debes tener los archivos `2-2ideal.asm`, `2-2data.asm`, `2-2contr-j.asm`, `2-2contr-bnez.asm` y `2-2struc.asm`.

Ejercicios

Habrás observado que en todos los programas de la sesión aparecen instrucciones `nop`. Esto es así para evitar riesgos adicionales que dificultasen la comprensión de cada tipo de riesgo en cada uno de los apartados. Prueba a eliminar las instrucciones `nop` de los programas y a simularlos de nuevo. Observarás que aparecen nuevos riesgos que antes no aparecían al introducir instrucciones *inútiles* en la ejecución del código *útil* de los programas. ¿Coinciden los resultados con los obtenidos anteriormente?